

Funzioni in C++

Funzioni in C++

Come definire e usare le funzioni in C++.

Cos'è una funzione?

Immagina di dover calcolare l'area di un cerchio in dieci punti diversi del tuo programma. Scriveresti la stessa formula dieci volte? E se poi vuoi cambiarla, la aggiorni in dieci posti?

Le **funzioni** risolvono questo problema: scrivi il codice una volta, gli dai un nome, e lo richiami ogni volta che ne hai bisogno. È come salvare una ricetta: la scrivi una volta, poi la segui ogni volta che vuoi cucinare quel piatto.

Definire una funzione

Una funzione si scrive così:

```
tipo_ritorno nomeFunzione(tipo parametro1, tipo parametro2) {  
    // codice della funzione  
    return valore; // se il tipo di ritorno non è void  
}
```

La funzione più semplice: non riceve nulla e non restituisce nulla:

```
void saluta() {  
    cout << "Ciao!" << endl;  
}
```

void significa "niente": questa funzione non restituisce nessun valore.

Chiamare una funzione

Una volta definita, puoi "chiamarla" (cioè eseguirla) scrivendo il suo nome seguito dalle parentesi:

```
void saluta() {  
    cout << "Ciao!" << endl;  
}  
  
int main() {  
    saluta();    // esegue la funzione – stampa "Ciao!"  
    saluta();    // la puoi chiamare quante volte vuoi  
    return 0;  
}
```

Passare informazioni: i parametri

I **parametri** sono informazioni che puoi passare alla funzione quando la chiami:

```
// Questa funzione riceve un nome e lo usa per salutare  
void saluta(string nome) {  
    cout << "Ciao, " << nome << "!" << endl;  
}  
  
int main() {  
    saluta("Alice");    // stampa: Ciao, Alice!  
    saluta("Bob");      // stampa: Ciao, Bob!  
    return 0;  
}
```

Puoi avere più parametri separati da virgola:

```
// Riceve base e altezza, calcola area e perimetro
void stampaRettangolo(int base, int altezza) {
    cout << "Area: " << base * altezza << endl;
    cout << "Perimetro: " << 2 * (base + altezza) << endl;
}

int main() {
    stampaRettangolo(5, 3);
    stampaRettangolo(10, 7);
    return 0;
}
```

Ricevere un risultato: il valore di ritorno

Una funzione può anche **restituire** un valore al codice che la ha chiamata. Si usa `return` :

```
// Riceve due numeri e restituisce la loro somma
int somma(int a, int b) {
    return a + b;
}

int main() {
    int risultato = somma(3, 5);
    cout << risultato << endl;    // 8

    // Puoi usare la funzione direttamente nelle espressioni
    cout << somma(10, 20) * 2 << endl;    // 60

    return 0;
}
```

Quando il compilatore incontra `return` , la funzione si ferma subito e restituisce il valore.

Il prototipo: dichiarare prima di usare

In C++, una funzione deve essere **dichiarata prima del punto in cui la chiami**. Se vuoi definire le funzioni dopo il `main` , devi prima scrivere il loro **prototipo** (una dichiarazione anticipata):

```
// Prototipo – dice al compilatore che questa funzione esisterà
int somma(int a, int b);

int main() {
    cout << somma(3, 5) << endl;    // funziona!
    return 0;
}

// Definizione completa – scritta dopo il main
int somma(int a, int b) {
    return a + b;
}
```

Il prototipo ha la stessa firma della funzione ma termina con `;` invece del corpo.

Valori di default

Puoi dare un valore predefinito a un parametro. Se chi chiama la funzione non lo specifica, viene usato il valore di default:

```
void saluta(string nome, string saluto = "Ciao") {
    cout << saluto << ", " << nome << "!" << endl;
}

int main() {
    saluta("Alice");                // usa il default – stampa: Ciao, Alice!
    saluta("Bob", "Buongiorno");    // usa il valore specificato – stampa:
    Buongiorno, Bob!
    return 0;
}
```

I parametri con valori di default devono stare sempre in fondo alla lista.

Più istruzioni `return`

Una funzione può avere più punti di uscita. Appena il programma incontra `return`, la funzione si ferma:

```
string classificaVoto(int voto) {  
    if (voto >= 9) return "Ottimo";  
    if (voto >= 7) return "Buono";  
    if (voto >= 6) return "Sufficiente";  
    return "Insufficiente"; // raggiunto solo se nessun return precedente  
    è scattato  
}
```

Passaggio per valore: la funzione riceve una copia

Di default, una funzione riceve una **copia** del valore. Le modifiche che fa non cambiano la variabile originale:

```
void raddoppia(int x) {  
    x = x * 2;    // modifica solo la copia - l'originale non cambia  
    cout << "Dentro la funzione: " << x << endl;  
}  
  
int main() {  
    int numero = 5;  
    raddoppia(numero);  
    cout << "Fuori dalla funzione: " << numero << endl; // ancora 5!  
    return 0;  
}
```

Output:

```
Dentro la funzione: 10  
Fuori dalla funzione: 5
```

Per modificare la variabile originale, si usano i riferimenti (trattati nella sezione memoria).

Esempio pratico

```
#include <iostream>
using namespace std;

// Calcola l'area di un cerchio dato il raggio
double calcolaAreaCerchio(double raggio) {
    const double PI = 3.14159265358979;
    return PI * raggio * raggio;
}

// Restituisce true se il numero è pari, false altrimenti
bool isPari(int numero) {
    return numero % 2 == 0;
}

// Restituisce il maggiore dei due numeri
int massimo(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    double raggio = 5.0;
    cout << "Area cerchio (r=" << raggio << "): " <<
    calcolaAreaCerchio(raggio) << endl;

    int n = 7;
    cout << n << " è " << (isPari(n) ? "pari" : "dispari") << endl;

    cout << "Massimo tra 15 e 23: " << massimo(15, 23) << endl;

    return 0;
}
```